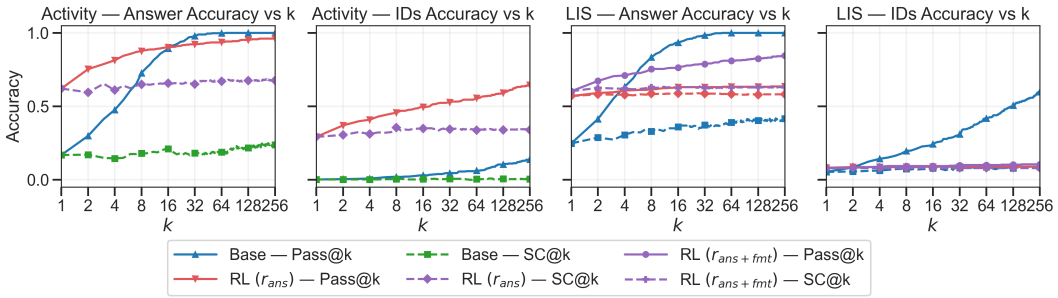

Figure 2: Performance comparison of Base, RL(r_{ans}), and RL($r_{\text{ans+fmt}}$) models on the *Activity* and *LIS* task with the Qwen2.5-7B model.

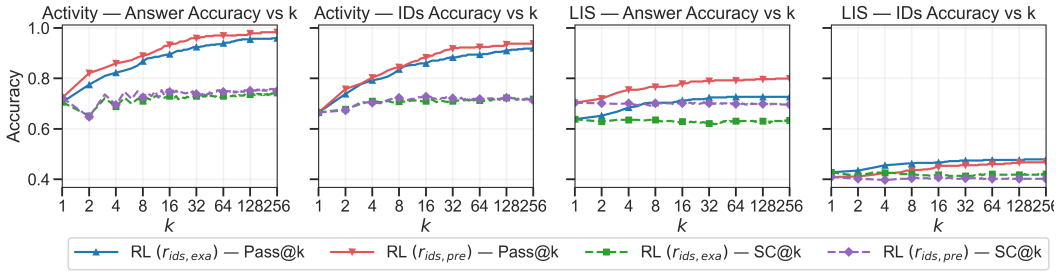

Figure 3: Performance comparison of RL models trained with $r_{ids,exa}$ and $r_{ids,pre}$.

slightly lower *Pass@k* at large k, it achieves much higher self-consistency (about 0.68 vs. ~ 0.24 at k=256), indicating more stable predictions across samples. Under exact sequence ID matching, RLVR substantially outperforms the base model: at k=256, RLVR reaches $\text{Pass@k} \approx 0.64$ compared to 0.14, with $\text{SC} \approx 0.34$ compared to 0.004, reflecting a clear improvement in sequence-level fidelity. These results support prior findings that answer-only *Pass@k* can overstate model capability [16], whereas Acc_{ids} and SC provide more faithful measures of reasoning quality. RLVR improves both by reinforcing verified reasoning trajectories [16].

LIS. On *LIS*, training with the answer-only reward r_{ans} rapidly collapses intermediate reasoning: after just a few PPO updates, the policy drops its chain of thought and outputs terse final answers, reflected in a sharp decline in mean response length (Appendix D). Adding a format component ($r_{\text{ans+fmt}}$) mitigates this, keeping response lengths closer to the base model (Fig. 7). As shown in Figure 2, both RLVR-trained models achieve higher SC on answer than the base model; for r_{ans}, the *Pass@k* and SC curves nearly overlap, whereas $r_{\text{ans+fmt}}$ underperforms the base model at larger k. However, under exact sequence ID evaluation, both RLVR-trained policies remain weak, with low *Pass@k* and SC compared to the base model, in contrast to the *Activity* task, where RLVR improved both metrics.

Takeaway 1

RLVR improves answer-level generalization on both *Activity Scheduling* and *LIS*. However, only in *Activity Scheduling* do we observe reasoning gains, while on *LIS*, improvements stem from superficial heuristics or formatting strategies. Thus, RLVR can yield apparent task generalization without strengthening the underlying reasoning process.

3.2 Training with Sequence Rewards

We now evaluate sequence-aware objectives, comparing the exact-match reward $r_{\text{ids,exa}}$ and the prefix reward $r_{\text{ids,pre}}$ (cf. §2.2). Figure 3 summarizes results for Acc_{ans} and Acc_{ids} across k. On *Activity*, both rewards yield similar gains, substantially surpassing the base and r_{ans} models: at k=256, SC_{ids} rises from 0.34 to $0.72/0.71$ ($r_{\text{ids,exa}}/r_{\text{ids,pre}}$), and SC_{ans} increases from 0.68 to $0.74/0.75$. On *LIS*, a trade-off emerges: $r_{\text{ids,pre}}$ attains higher Acc_{ans}, while $r_{\text{ids,exa}}$ achieves higher Acc_{ids}, yet both outperform

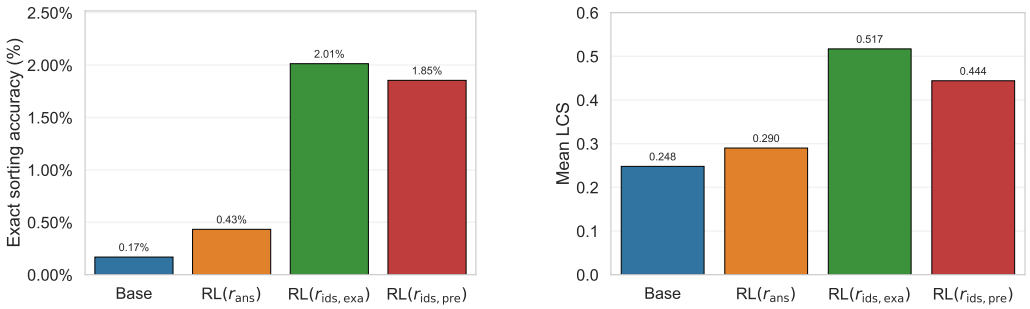

Figure 4: Sorting accuracy and LCS across models.

the base and r_{ans} policies. At $k{=}256$, SC_{ids} improves from ≈ 0.08 to $0.42/0.40$, and SC_{ans} from 0.58 to $0.63/0.70$, indicating that sequence rewards enhance both answer- and sequence-level consistency.

Takeaway 2

Sequence-aware rewards improve sequence fidelity on both *Activity Scheduling* and *LIS*, as reflected in higher Acc_{ids}. They also provide modest secondary gains in Acc_{ans}, suggesting that RLVR generalization can benefit from incorporating intermediate or auxiliary objectives.

3.3 Sorting Performance on Activity Task

Sequence-aware rewards improve sequence matching partly by enhancing the "sorting preface" that models generate as the first step in activity scheduling. We extract candidate ID lists from outputs using simple patterns, succeeding on over 84% of RL-trained responses versus 40% for the base model (Appendix B). Evaluating **ExactSort** (exact match with ground truth) and **LCS fraction** (longest correctly sorted subsequence) shows that sequence rewards yield both higher exact sorting and better partial order fidelity.

Figure 4 summarizes sorting quality for the four models. Both sequence-reward policies ($r_{\text{ids,exa}}$ and $r_{\text{ids,pre}}$) improve *exact sorting* relative to the base and r_{ans} models (from $0.17\%/0.43\%$ to $2.01\%/1.85\%$) and increase the mean LCS (from $0.248/0.290$ to $0.517/0.444$). However, absolute exact-sorting accuracy remains very low ($\approx 2\%$), even though these same policies achieve high sequence correctness at evaluation (e.g., $\text{Pass@256}_{\text{ids}} \approx 0.60$ on *Activity*). This gap indicates that the sorting step is not reliably driving the final schedule, despite the frequent appearance of a "sorted" preface.

3.4 Training with Sorting-Match Reward

In §3.2, we observed performance gains in Acc_{ans} when training with sequence-aware rewards. This raises the question of whether rewarding the sorting step, which is the first stage of the greedy activity-scheduling algorithm, could provide additional benefit. To test this, we trained a model with the sorting-match reward r_{sort}.

Surprisingly, this led to a catastrophic collapse: both Acc_{ans} and Acc_{ids} fell to nearly 0%. Inspection of the outputs revealed that the model consistently produced the sorted sequence itself as the final answer, without applying the non-overlap constraint required for valid schedules.

We next trained a model using a combined objective with equal weights on r_{ans}, r_{ids}, and r_{sort}. This configuration restored Acc_{ans} and Acc_{ids} to levels comparable to the r_{ids} model, but it did not improve sorting performance on the test set. Thus, learning to sort in isolation does not confer complementary benefits for solving the activity-scheduling task.

To probe further, we ran a curriculum experiment: training with r_{sort} alone for the first 10, 20, or 30 PPO updates, and then switching to r_{ans} for the remainder of the 120 training steps. As shown in Figure 5, models trained with r_{sort} for only 10 steps were able to recover under r_{ans}, but longer

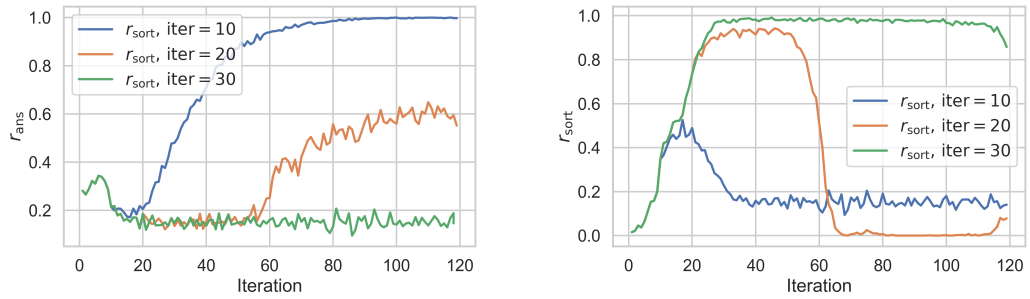

Figure 5: Curriculum experiments with r_{sort}. Each curve shows accuracy on the training set when models are trained with r_{sort} for the first 10, 20, or 30 PPO updates, followed by r_{ans} for the remainder. Longer pretraining with r_{sort} makes it increasingly difficult for the model to recover under r_{ans}.

exposure (20 or 30 steps) severely hindered recovery. In fact, after 30 steps of r_{sort}, the model failed to improve Acc_{ans} even on the training data.

Takeaway 3

In *Activity Scheduling*, RLVR with sequence rewards improves answer and sequence accuracy, but sorting accuracy remains very low. Models often emit a superficial "sorted" preface that neither matches the canonical order nor drives the final schedule, highlighting a disconnect between surface outputs and the underlying decision rule [12].

3.5 Heuristics Analysis for LIS

To probe what signals models exploit on *LIS*, we regress predicted numeric answers against human-interpretable features from the input table.

We pool all k stochastic runs but split train/test by problem instance ID to avoid leakage. Features cover: (i) global scale (length, range, dispersion, quantiles), (ii) order structure (increase ratios, inversion ratio, sign changes), (iii) run structure (longest runs, monotone counts, local extrema, record highs/lows), (iv) simple LIS heuristics (greedy, beam-limited, limited backtracking), and (v) patience-sorting tails. We train a Random Forest regressor and report R^2 and mean absolute error (MAE). Details appear in Appendix E.

Model	R^2_{test}	MAE_{test}
Base	-0.002	2.745
r_{ans}	0.741	0.269
$r_{\text{ids,exa}}$	0.781	0.289
$r_{\text{ids,pre}}$	0.841	0.227
$r_{\text{ans+fmt}}$	0.867	0.209

Table 1: Predictive fit of LIS features on model outputs.

Table 1 shows that RLVR-trained outputs are predicted well from these features, unlike the base model. While the features are not exhaustive, results suggest RLVR amplifies systematic heuristics aligned with task structure, boosting answer-level performance without ensuring stronger reasoning.

4 Limitations & Future Work

Our study investigates two verifiable reasoning tasks (*Activity Scheduling* and *LIS*) with a single base model (*Qwen2.5-7B-Instruct*). While this controlled setup helps isolate RLVR dynamics, conclusions may not generalize across tasks, model families, or scales, as prior work has shown diverse shortcut behaviors and model-dependent failure modes [7]. To partially address this, we provide additional results for *Llama-3.1-8B* in Appendix F.

Future work could expand to a broader range of models and problem domains, and employ mechanistic interpretability to probe the circuits and dynamics underlying RLVR-induced behaviors [9]. Such analysis may clarify whether improvements reflect genuine task learning or superficial strategies that exploit evaluation metrics. Our findings highlight this tension, emphasizing the need for careful evaluation and diagnostics when claiming improvements in reasoning from RLVR.